NXP Semiconductors
Application Note

Document Number: AN12149
Rev. 0, 03/2018

Implementing the IEEE 1588 V2 on i.MX RT Using PTPd, FreeRTOS, and lwIP TCP/IP stack

1. Introduction

This application note describes the implementation of the IEEE 1588 V2 Precision Time Protocol (PTP) on the i.MX RT processors running FreeRTOS. The IEEE 1588 standard provides accurate clock synchronization for distributed control nodes for industrial automation applications.

The implementation is running on the MIMXRT1050 Evaluation Kit (EVK-MIMXRT1050) with the MIMXRT1052DVL6A processor. The demo software is based on the NXP MCUXpresso SDK 2.3.0 release for EVK-MIMXRT1050. The demo is a PTP daemon (PTPd) using the lwIP TCP/IP stack shipped with the MCUXpresso SDK and runs on FreeRTOS. The PTP daemon is an open-source implementation of the PTP.

This document describes the IEEE 1588 protocol basics, IEEE 1588 functions on the i.MX RT MCUs, and detailed description of the IEEE 1588 demo software, including how to port the PTPd for FreeRTOS on the i.MX RT processors and how to enable the ENET output compare function to monitor the clock synchronization status. This document also describes how to build and run the demo.

Contents

2. IEEE 1588 basic overview

The IEEE 1588 standard is called the Precision Clock Synchronization Protocol for Networked Measurement Control Systems, also known as PTP. The IEEE 1588 PTP enables the clocks distributed across an Ethernet network to be accurately synchronized using a process where the distributed nodes exchange timestamped messages.

The technology behind the standard was originally developed by Agilent Technologies and used for distributed measuring and control tasks. The challenge is to synchronize the networked measuring devices to each other in terms of time, enabling them to record the measured values and providing them with a precise system timestamp. Based on this timestamp, the measured values can then be correlated to each other.

The typical applications of the IEEE 1588 time synchronization include:

- Time-sensitive telecommunication services that require precise time synchronization between the communicating nodes.
- Industrial network switches that synchronize sensors and actuators over a single-wire distributed control network to control an automated assembly process.
- Powerline networks that synchronize across large-scale distributed power-grid switches to enable a smooth power transfer.
- Test/measurement devices that must maintain accurate time synchronization with the device under the test in many different operation environments.
- Printing machines, cooperative robotic systems, and residential Ethernet.

These applications require precise clock synchronization between the devices with an accuracy in the sub-microsecond range. It is a remarkable feature of the IEEE 1588 that this synchronization precision is achieved using regular Ethernet connectivity with standard Ethernet frames.

This solution enables nearly any device of any performance to participate in high-precision synchronized networks that are simple to operate and configure.

The other key benefits of the IEEE 1588 protocol include:

- Convergence times shorter than a minute for sub-microsecond synchronization between heterogeneous distributed devices with different clocks, resolution, and stability.
- Automatic configuration and segmentation. Each node uses the Best Master Clock (BMC) algorithm to determine the best clock in the segment. Every PTP node stores its features within a specified dataset. These features are transmitted to other nodes within their sync telegrams. Based on this, the other nodes can synchronize their data sets with the features of the actual master and adjust their clocks. The cyclic running of the BMC also enables hot swapping; that is, the nodes can be connected or removed during propagation time.
- Simple configuration and operation with low computing requirements and network bandwidth consumption.

2.1. Synchronization principle

The network clocks are organized in a master-slave hierarchy. IEEE 1588 identifies the master clock and then establishes a two-way timing exchange by which the master sends messages to its slaves to initiate the synchronization. Each slave then responds to synchronize itself to its master. This sequence is repeated throughout the specified network to achieve and maintain the clock synchronization.

The process starts with one node (master clock) transmitting a sync telegram that contains the estimated transmission time. The exact transmission time of the sync telegram is captured by a clock and transmitted in a second follow-up message. By comparing the timestamp information contained within the first and second telegrams against its own clock, the receiver can calculate the time difference between its own clock and the master clock (see Figure 1). The sync and follow-up messages are sent as a multicast. Some IEEE 1588 systems enable the hardware timestamping and the insertion of actual timestamps into the sync message. In this case, the follow-up messages are not needed (one-step mode of operation).

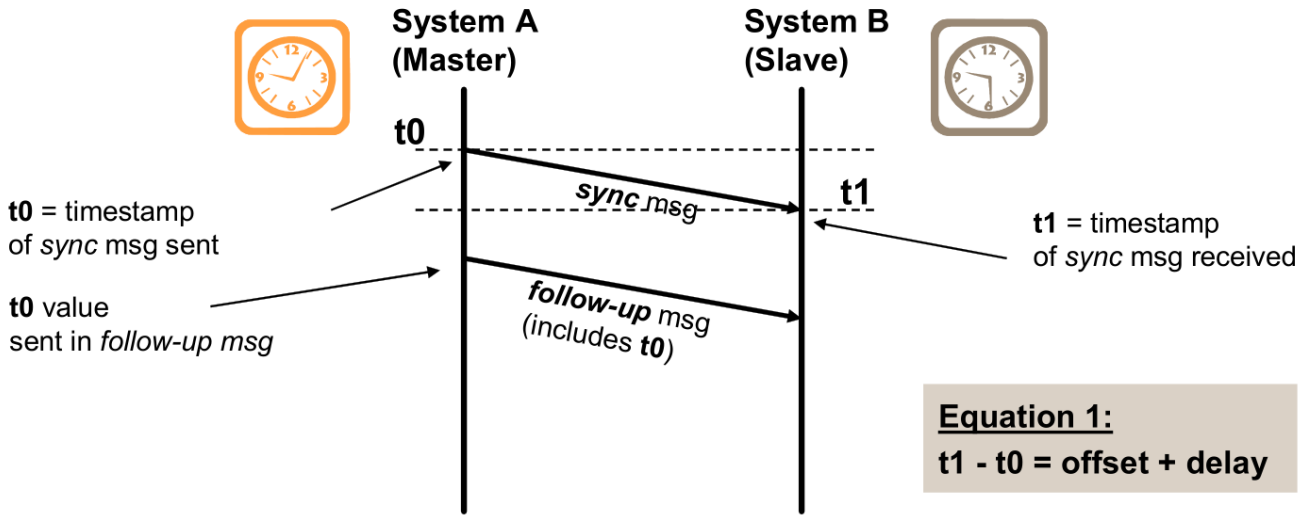

Figure 1. Offset and delay measurement—sync message, follow-up message

The telegram propagation time is determined cyclically in the second transmission process between the slave and the master (delay telegrams). The slave then corrects its clock and adapts it to the current bus propagation time (see Figure 2). The delay_req and delay_resp messages are point-to-point, but sent with a multicast address for simplicity reasons.

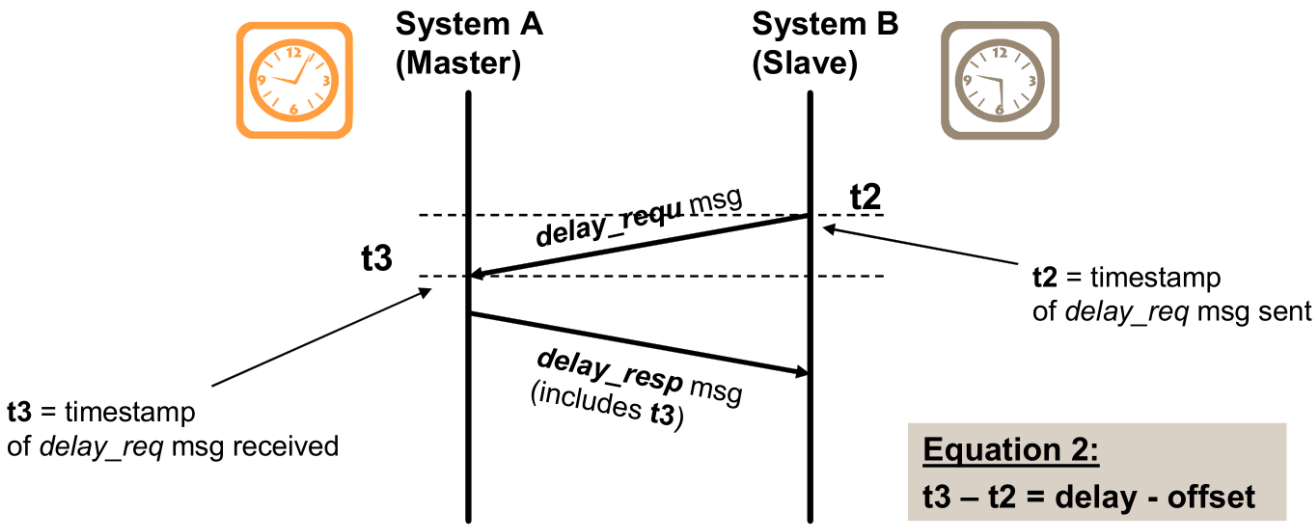

Figure 2. Offset and delay measurement—delay messages

Figure 3 shows an example of the IEEE 1588 synchronization sequence (one cycle) and the calculation of the actual offset and the actual delay between the master and slave nodes.

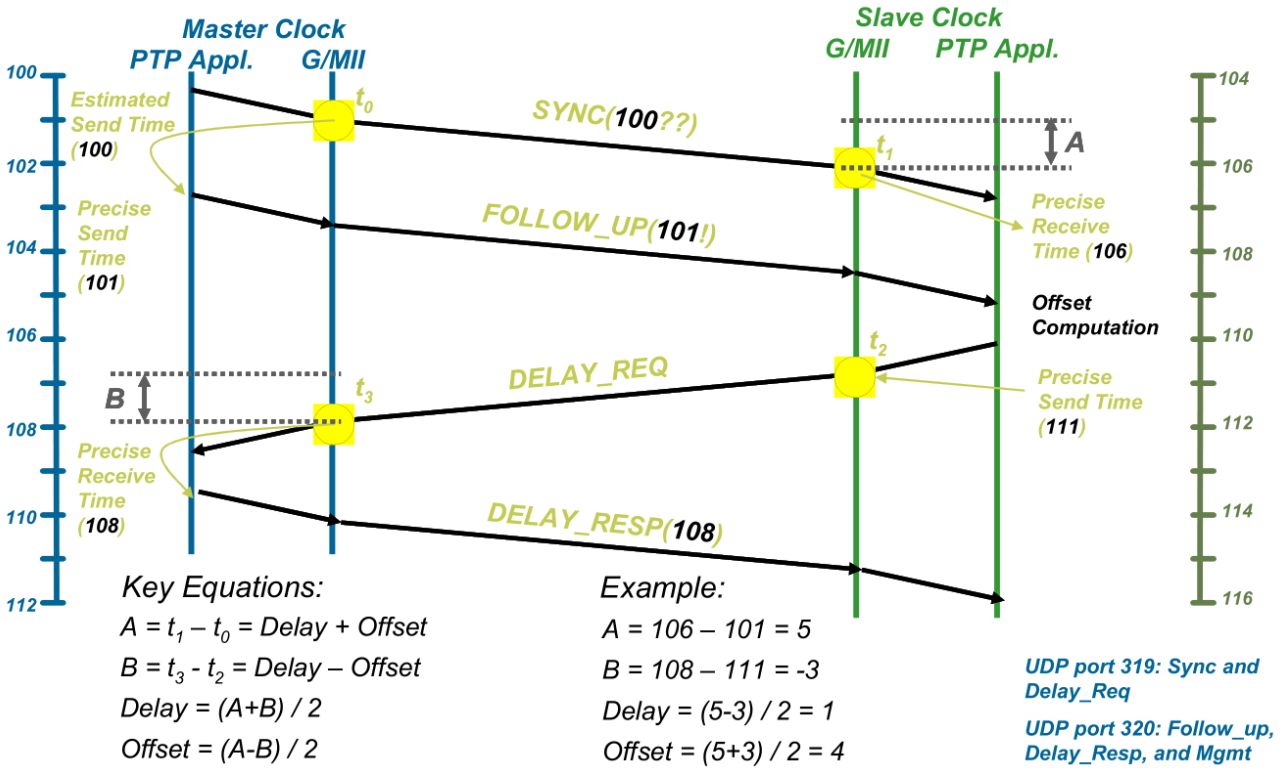

Figure 3. IEEE 1588 synchronization message sequence

For more information about the IEEE 1588 standard, visit the web page of the National Institute of Standards and Technology.

2.2. Timestamping

The PTP protocol can be completely implemented into the software using a standard Ethernet module. Because the timestamp information is applied at the application level, the delay fluctuation introduced by the software stack running on the master and slave devices means that only a limited precision can be achieved (see Figure 4).

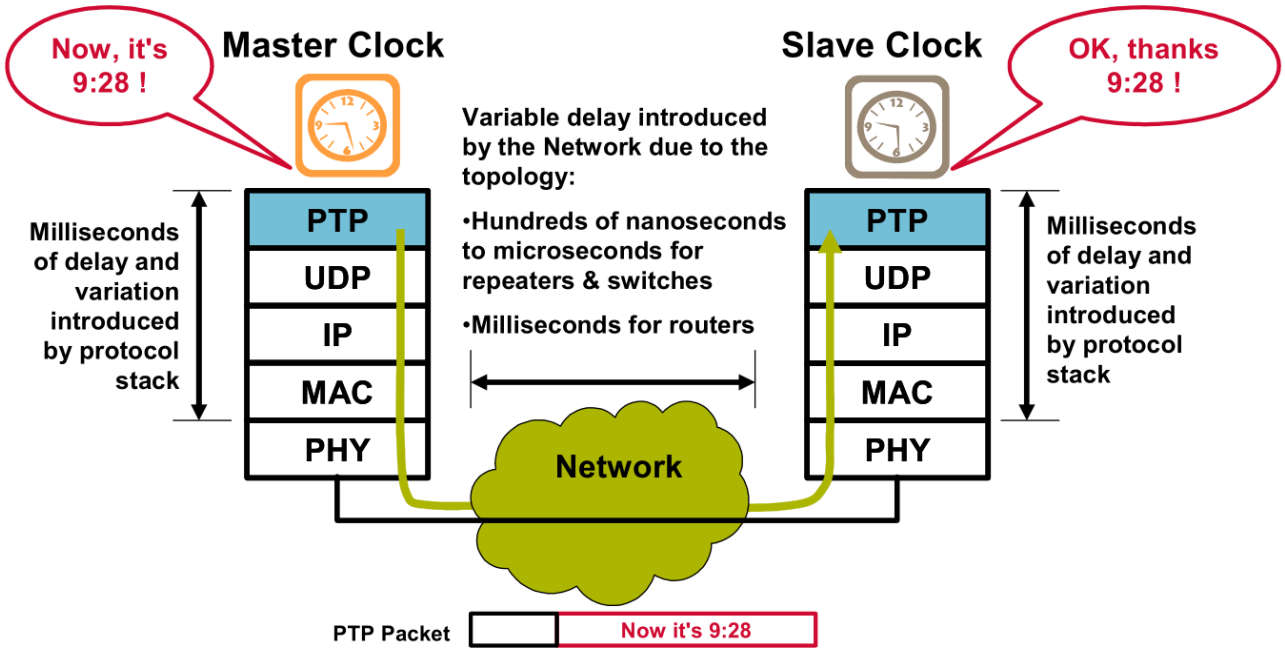

Figure 4. Software timestamp implementation

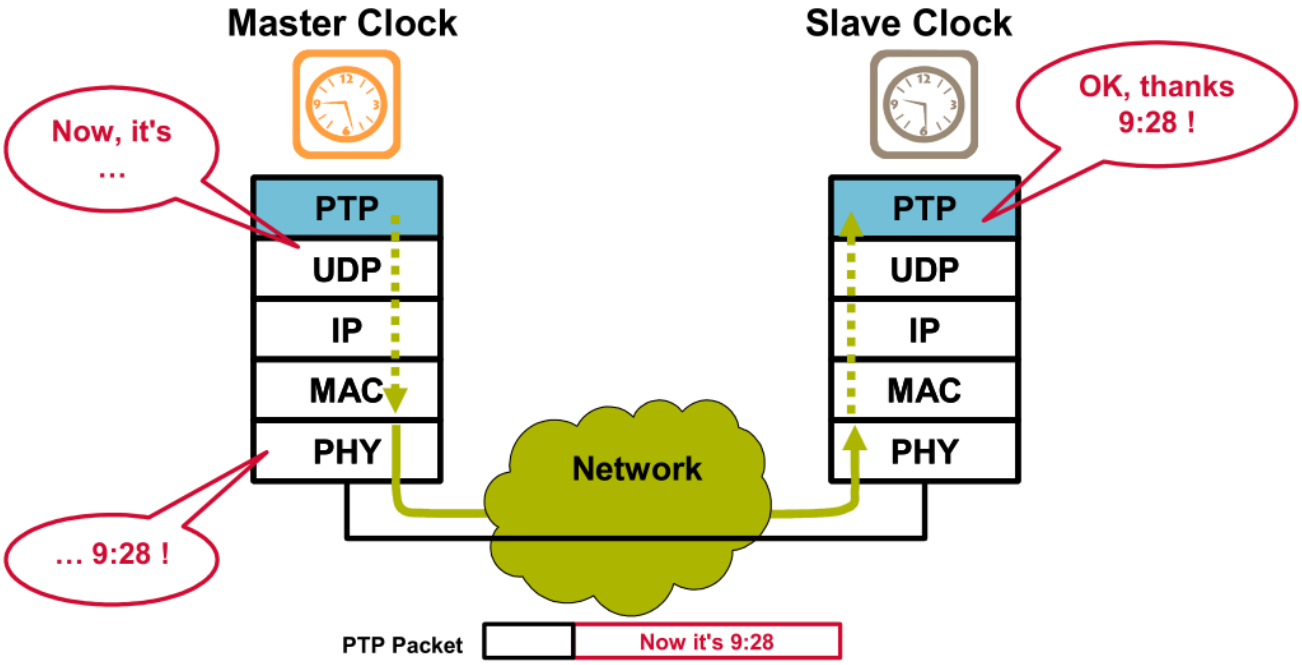

Figure 5. Hardware timestamp implementation

To minimize the impact of the protocol stack delay, take the timestamps closer to the physical interface; that is, to the MAC or PHY layers (see Figure 5). Dedicated hardware with timestamping capabilities, such as the MAC-NET peripheral module, 10/100-Mbps Ethernet MAC (ENET), or NXP i.MX RT 1050 and 1020 allows for synchronization with significantly improved accuracy.

3. IEEE 1588 functions on i.MX RT

The i.MX RT integrates the MAC-NET core in conjunction with a 10/100-Mbit/s MAC to accelerate the processing of various common networking protocols (such as IP, TCP, UDP, and ICMP), providing wire speed services to client applications. The unified DMA (uDMA), which is internal to the ENET module, optimizes the data transfer between the ENET core and the SoC and supports an enhanced buffer descriptor programming model to support the IEEE 1588 functionality. To enable the IEEE 1588 or similar time-synchronization protocol implementations, the MAC is combined with a timestamping module to support precise timestamping of incoming and outgoing frames. Set the EN1588 bit in the ENET_ECR (Ethernet Control Register) to enable the 1588 support.

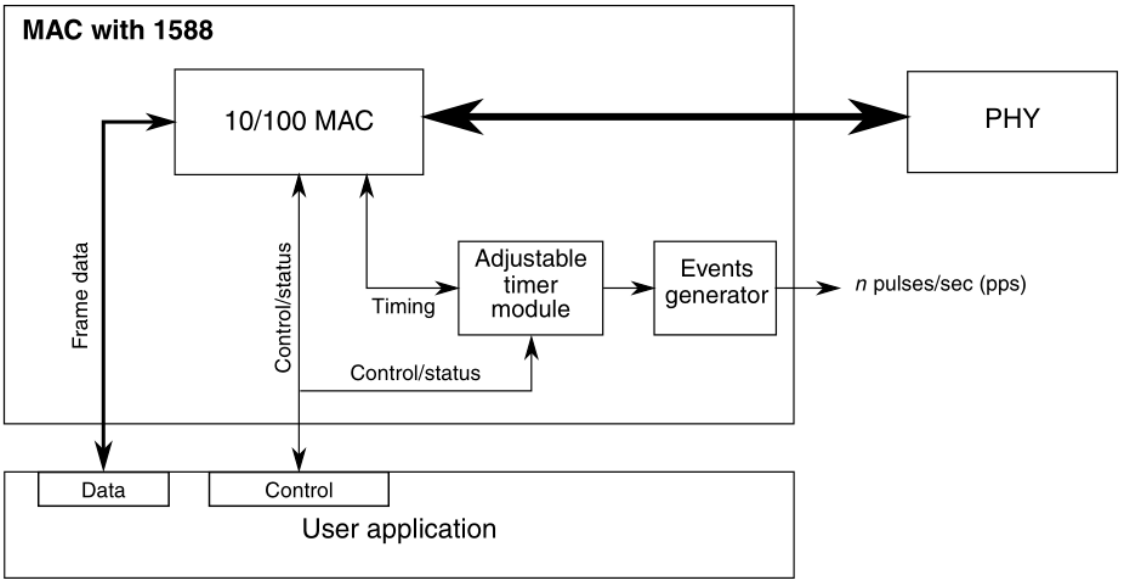

Figure 6. IEEE 1588 functions overview

3.1. Adjustable timer module

The adjustable timer module (TSM) implements the Free-Running Counter (FRC), which generates the timestamps. The FRC operates with the timestamping clock, which can be set to any value, depending on the system requirements.

Through a dedicated correction logic, the timer can be adjusted to enable synchronization to a remote master and provide a synchronized timing reference to the local system. The timer can be configured to trigger an interrupt after a fixed time period to enable the synchronization of software timers or perform other synchronized system functions.

The timer is typically used to implement a period of one second; its value ranges from 0 to $(1 \times 10^9) - 1$. The period event can trigger an interrupt, and the software can maintain the second and hour time values as necessary.

The adjustable timer consists of a programmable counter/accumulator and a correction counter. The periods of both counters and their increment rates are freely configurable, enabling a very fine tuning of the timer.

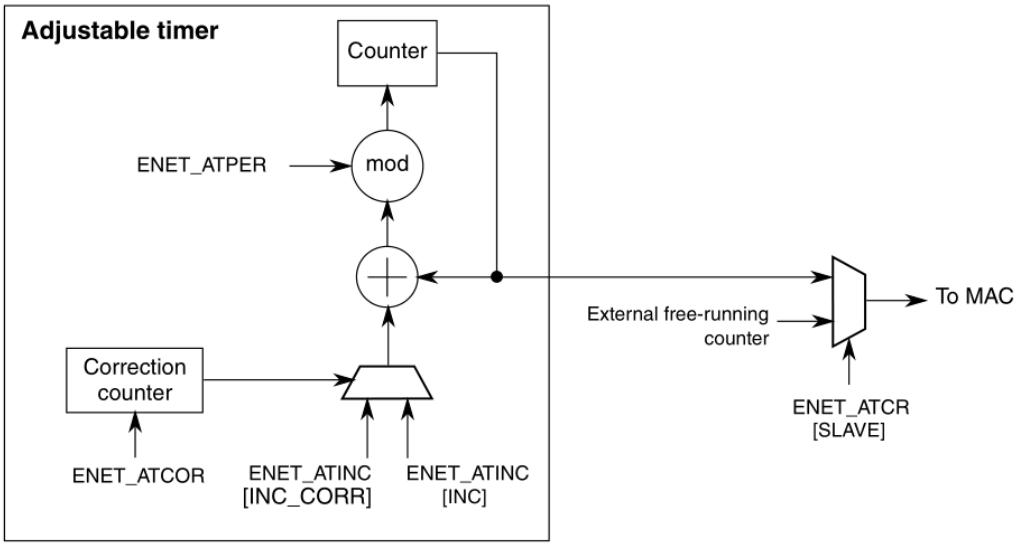

Figure 7. Adjustable timer implementation detail

The counter produces the current time. During every timestamping clock cycle, a constant value is added to the current time, as programmed in ENET_ATINC (Timestamping Clock Period Register). The value depends on the chosen timestamping clock frequency. For example, if it operates at 125 MHz, setting the increment to 8 represents 8 ns.

The period, configured in ENET_ATPER (Timer Period Register), defines the modulo when the counter wraps. In a typical implementation, the period is set to 1×10^9 so that the counter wraps every second and all timestamps represent the absolute value in nanoseconds within a one-second period. When the period is reached, the counter wraps to start again, respecting the period modulo. This means it does not necessarily start from zero, but the counter is loaded with the value of (Current + Inc - (1×10^9)), assuming the period is set to 1×10^9.

The correction counter is completely independent and increments by one with each timestamping clock cycle. When it reaches the value configured in ENET_ATCOR (Timer Correction Register), it restarts and instructs the timer once to increment by the correction value, instead of the normal value.

The normal and correction increments are configured in ENET_ATINC. To speed up the timer, set the correction increment to more than the normal increment value. To slow down the timer, set the correction increment to less than the normal increment value.

The correction counter only defines the distance of the corrective actions, not the amount. This allows for very fine corrections and low jitter (in the range of 1 ns), independent of the chosen clock frequency.

3.2. Transmit timestamping

Only 1588 event frames must be timestamped on transmit. The client application (for example, the MAC driver) should detect 1588 event frames and set the TS bit in the TxBD (Enhanced Transmit Buffer Descriptor) together with the frame.

If the TxBD[TS] is set, the MAC records the timestamp for the frame in the ENET_ATSTMP (Timestamp of Last Transmitted Frame Register). The TS_AVAIL bit in the ENET_EIR (Interrupt Event Register) is set to indicate that a new timestamp is available.

The software implements a handshaking procedure by setting the TxBD[TS] when it transmits the frame for which a timestamp is needed and then waits for the ENET_EIR[TS_AVAIL] to determine when the timestamp is available. The timestamp is then read from the ENET_ATSTMP register. This is done for all event frames. The other frames do not use the TxBD[TS] and do not interfere with the timestamp capture.

3.3. Receive timestamping

When a frame is received, the MAC latches the value of the timer when the frame's Start of Frame Delimiter (SFD) field is detected and provides the captured timestamp in the 1588 timestamp field defined in the RxBD (Enhanced uDMA Receive Buffer Descriptor). This is done for all received frames.

3.4. Time synchronization

The adjustable timer module is available to synchronize the local clock of a node to a remote master. It implements a free-running 32-bit counter and also contains an additional correction counter.

The correction counter increases or decreases the rate of the free-running counter, allowing for very fine granular changes of the timer for synchronization, adding only a very low jitter when performing corrections.

In the slave scenario, the application software implements the required control algorithm by setting the correction to compensate for local oscillator drifts and locking the timer to the remote master clock on the network.

The timer and all timestamp-related information should be configured to show the true value (in nanoseconds) in a period of 1 s (the timer is configured to have a period of 1 s). The values range from 0 to $(1 \times 10^9) - 1$. In this application, the seconds counter is implemented in the software using an interrupt function which is executed when the nanoseconds counter wraps at 1×10^9.

3.5. Input capture and output compare

The input capture and output compare block can be used to provide a precise hardware timing for the input and output events. The IEEE 1588 timer has four channels. Each channel supports the input capture and output compare using the 1588 counter.

In the input capture mode, the TCCRn (Timer Compare Capture Register, n = 1, 2, 3, 4) latches the time value when the corresponding external event occurs. The event can be a rising-, falling-, or either-edge of one of the 1588_TMRn signals. The event sets the corresponding TCSRn[TF] (Timer Control Status

Register) timer flag, indicating that an input capture occurred. If the corresponding interrupt is enabled in the TCSRn[TIE] field, an interrupt can be generated.

In the output compare mode, the TCCRn compare registers are loaded with the time at which the corresponding event should occur. When the ENET free-running counter value matches the output compare reference value in the TCCRn register, the corresponding flag (TCSRn[TF]) is set, indicating that an output compare occurred. The corresponding interrupt (if enabled by the TCSRn[TIE]) is generated. The corresponding 1588_TMRn output signal is asserted according to the TCSRn[TMODE].

4. IEEE 1588 implementation for i.MX RT

The MIMXRT1050 Evaluation Kit (EVK-MIMXRT1050) is used as the hardware platform for the hardware timestamping-based IEEE 1588 V2 Precision Time Protocol. The solution uses the MCUXpresso SDK for EVK-MIMXRT1050, which includes the NXP ENET driver of the i.MX RT MPU, the PHY driver, the ported FreeRTOS, and the ported lwIP TCP/IP stack for the EVK-MIMXRT1050 board. The IEEE1588 V2 Precision Time Protocol is implemented by the PTP daemon application, which is an open-source implementation of the Precision Time Protocol. Figure 8 shows the hardware and software components of this solution.

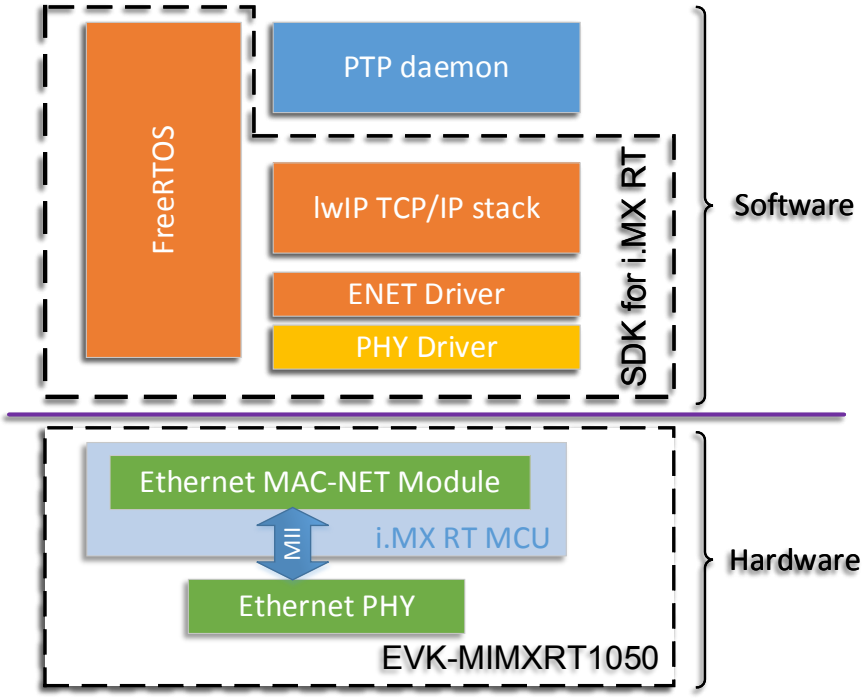

Figure 8. IEEE 1588 solution for i.MX RT processor

4.1. Hardware components

The MIMXRT1050 Evaluation Kit (EVK-MIMXRT1050) is the platform designed to showcase the most common features of the i.MX RT 1050 processor. The EVK-MIMXRT1050 is an entry-level development board which helps the developers to become familiar with the processor quickly and

expedites the customers to implement their own elaborate designs. The main features of the EVK-MIMXRT1050 include:

- MIMXRT1052DVL6A MCU
- 32 MB @ 166 MHz SDRAM
- 512-Mbit hyper flash, +64-Mbit quad SPI flash, and TF card slot
- 10/100-Mbit/s Ethernet connector with KSZ8081RNB PHY
- USB 2.0 OTG/host connectors
- 3.5-mm audio stereo headphone jack, microphone, and speaker output connectors
- Display connector and CMOS sensor interface
- CAN bus connector, OpenSDA with DAP-Link, and Arduino interface
- 5 V DC jack for the power supply

The i.MX RT 1050 is the first crossover processor in the industry, which is a new processor family featuring NXP's advanced implementation of the ARM® Cortex®-M7 core. It is designed to support the next generation of IoT applications with a high level of integration and security balanced with the MCU-level usability. It operates at speeds of up to 600 MHz to provide high CPU performance with the best real-time functionality. The i.MX RT 1050 provides various memory interfaces, including SDRAM, raw NAND flash, NOR flash, SD/eMMC, quad SPI, HyperBus, and a wide range of other interfaces for connecting peripherals, such as WLAN, Bluetooth™, GPS, display, and camera sensors. As the other i.MX processors, the i.MX RT1050 also integrates rich audio and video features, including LCD display, basic 2D graphics, camera interface, SPDIF, and I^2S audio interfaces.

Figure 9 shows a block diagram of the i.MX RT 1050 MCU. For more information, see the *i.MX RT1050 Processor Reference Manual* (document IMXRT1050RM).

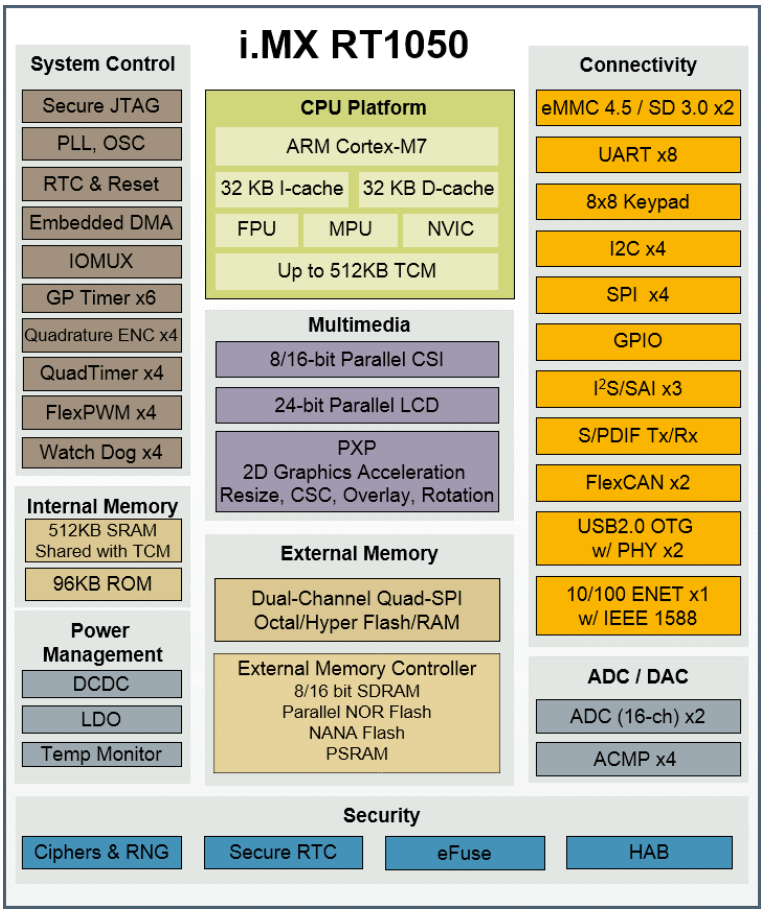

Figure 9. i.MX RT 1050 block diagram

4.2. Software components

The software of the IEEE 1588 implementation includes the MCUXpresso SDK for the EVK-MIMXRT1050 and the PTP daemon. The MCUXpresso SDK is a software framework for developing applications on the NXP MCUs, including peripheral drivers, middleware, and real-time operating system.

4.2.1. FreeRTOS real-time operation system

The FreeRTOS is a real-time kernel (or real-time scheduler) on top of which you can build embedded applications that meet strict real-time requirements. The FreeRTOS provides methods for multiple tasks, mutexes, semaphores, and software timers. A tickles mode is provided for low-power applications. The thread priorities used by the scheduler to decide which thread should be executing are supported. The FreeRTOS provided by the MCUXpresso SDK for EVK-MIMXRT1050 is version 9.0.0. The FreeRTOS package integrated in the MCUXpresso has these features:

- Removed the files not related to SDK (such as the extensions to the FreeRTOS (CLI, FAT_SL, and UDP)) and the folders (such as the *demo* folder and the nested folders).

- Added the *SystemCoreClock* global variable to the FreeRTOS *port.c* and *FreeRTOSConfig.h* files.
- Enabled the tickles mode. For more information, see www.nxp.com/freertos.
- Enabled the KDS Task Aware Debugger. Apply the FreeRTOS patch to enable the *configRECORD_STACK_HIGH_ADDRESS* macro.
- Enabled the *-flto* optimization in GCC by adding *__attribute__((used))* for *vTaskSwitchContext*.

For detailed information about FreeRTOS and its distribution, see www.freertos.org.

4.2.2. lwIP TCP/IP stack

The lwIP is a light-weight implementation of the TCP/IP protocol suite that is freely available in the C source code and can be downloaded from the development homepage. It is designed to be completely modular and small enough to reduce the RAM usage for use in small embedded systems. The core stack is an IP implementation, on top of which you can choose to add the TCP, UDP, DHCP, and many other protocols according to your needs and the memory size available in the designed system. For more information about the lwIP, see www.nongnu.org/lwip.

The MCUXpresso SDK for EVK-MIMXRT1050 integrates the lwIP TCP/IP stack, which runs on top of the MCUXpresso SDK Ethernet driver with the EVK-MIMXRT1050 board. The lwIP package version in the SDK for EVK-MIMXRT1050 is version 2.0.2. For more information, see the *lwIP TCP/IP Stack and MCUXpresso SDK Integration User's Guide* (document KSDKLWIPUG).

4.2.3. PTP daemon

The Precision Time Protocol (PTP) provides a precise time coordination of the Ethernet LAN-connected computers, which is designed primarily for the instrumentation and control systems. The PTP daemon (PTPd) is an open-source implementation for the PTP version 2, as defined by IEEE Std 1588-2008.

The PTP daemon can coordinate the clocks of a group of LAN-connected computers. It can achieve microsecond-level coordination, even on the limited platform. The PTPd is available in the C source code and easy to port on the FreeRTOS for an embedded system. Most system-related codes are in the *<install_dir>/src/dep* folder. The PTPd package version used in this demo is version 2.2.2, available at github.com/ptpd/ptpd/releases.

5. Detailed description of the IEEE1588 demo software

The demo application is currently compiled and tested only with the IAR Embedded Workbench® for ARM 8.20. The SDK version is SDK 2.3.0 and the PTPd version is 2.2.2. The i.MX RT ENET supports IEEE 1588 with a hardware timestamp. To enable the hardware timestamp feature and run the demo on the EVK-MIMXRT1050 board, the original lwIP TCP/IP port-related code must be updated. The update for the ENET driver is needed to test the clock offset converging range and for the bug fixes related to the IEEE 1588 functions.

5.1. i.MX RT SDK ENET driver update

The ENET driver update includes bug fixes and enables the ENET output compare function for test purposes. The output compare function is enabled to monitor the converging of the local slave clock offset to the remote master Ethernet LAN-connected i.MX RT 1050.

To get a correct transmit timestamp, add the following code in red into the *ENET_StoreTxFrameTime* function in the *fsl_enet.c* file.

```
Static status_t ENET_StoreTxFrameTime(ENET_Type *base, enet_handle_t *handle, uint32_t
ringId)
{
    ……
    if (isPtpEventMessage)
    {
        do
        {
            ……

            /* Do time stamp check on the last buffer descriptor of the frame. */
            if (curBuffDescrip->control & ENET_BUFFDESCRIPTOR_TX_LAST_MASK)
            {
                ……

                /* Get transmit time stamp second. */
                if (…)
                {
                    ptpTimeData.timeStamp.second = handle->msTimerSecond;
                }
                else
                {
                    ptpTimeData.timeStamp.second = handle->msTimerSecond - 1;
                }
                ptpTimeData.timeStamp.nanosecond = curBuffDescrip->timestamp;

                /* Enable the interrupt. */
                EnableGlobalIRQ(primask);
            }
            ……
        } while (handle->txBdDirtyTime[ringId] != handle->txBdCurrent[ringId]);
        return kStatus_ENET_TxFrameFail;
    }
    else
    {
        /* Increase current buffer descriptor to the next one. */
        if (handle->txBdDirtyTime[ringId]->control & ENET_BUFFDESCRIPTOR_TX_WRAP_MASK)
        {
            handle->txBdDirtyTime[ringId] = handle->txBdBase[ringId];
        }
        else
        {
            handle->txBdDirtyTime[ringId]++;
        }
    }
    return kStatus_Success;
}
```

The ENET 1588 timer has four channels, which support the input capture and output compare using the 1588 counter. To monitor the synchronicity between the master clock and the slave clock while the demo is running, the ENET output compare feature must be enabled to generate a Pulse-Per-Second

(PPS) signal while the free-running counter value matches the output compare reference value. If the clocks of the master and the slave are synchronized well and the output compare reference values of the master and the slave are set the same, the synchronization of the 1588 timer output signals of the master and the slave can be observed using an oscilloscope.

The code changes for the output compare enable in *fsl_enet.h* include an additional *enum* value in the *enum* type *enet_event_event* definition and additional two members in the *struct_enet_handle* type:

The code in red must be added or modified.

```
typedef enum _enet_event
{
   ……
   kENET_TimeStampAvailEvent, /*!< Timestamp available event.*/
   kENET_TimeStampCaptureEvent /*!< Timestamp capture event. */
} enet_event_t
```

```
struct _enet_handle
{
   ……
#ifdef ENET_ENHANCEDBUFFERDESCRIPTOR_MODE
   enet_ptp_time_data_ring_t txPtpTsDataRing;
   enet_ptp_timer_channel_t mPtpTmrChannel; /*!< PTP 1588 timer channel. */
   uint32_t ptpNextCounter;                 /*!< PTP 1588 next output compare counter value */
#endif                                      /* ENET_ENHANCEDBUFFERDESCRIPTOR_MODE */
};
```

To enable the output compare feature in the IEEE 1588 timer, the below code in red must be added or modified in the *ENET_Ptp1588Configure* and *ENET_Ptp1588TimerIRQHandler* functions.

```
void ENET_Ptp1588Configure(ENET_Type *base, enet_handle_t *handle, enet_ptp_config_t
*ptpConfig)
{
    ……
    handle->msTimerSecond = 0;
    handle->mPtpTmrChannel = ptpConfig->channel;
    /* Set the IRQ handler when the interrupt is enabled. */
    s_enetTxIsr = ENET_TransmitIRQHandler;
    ……
}
```

```
void ENET_Ptp1588TimerIRQHandler(ENET_Type *base, enet_handle_t *handle)
{
    ……
    else if (kENET_TsAvailInterrupt & base->EIR)
    {
       ……
    }
    else if (base->CHANNEL[handle->mPtpTmrChannel].TCSR & ENET_TCSR_TF_MASK)
    {
        ENET_Ptp1588SetChannelCmpValue(base, handle->mPtpTmrChannel, handle->ptpNextCounter);
        do {
            ENET_Ptp1588ClearChannelStatus(base, handle->mPtpTmrChannel);
        } while (true == ENET_Ptp1588GetChannelStatus(base, handle->mPtpTmrChannel));
    }
    ……
}
```

5.2. lwIP TCP/IP porting update

This section describes the modifications of the lwIP porting code to support the PTP demo. The affected files include the *lwipopts.h*, *ethernetif.h*, and *ethernetif.c* files in the *<sdk_install_dir>/middleware/lwip/port* folder.

The PTP daemon demo uses the *SO_REUSEADDR* option for the socket, DNS (Domain Name System) protocol, IGMP (Internet Group Management Protocol) protocol, and doesn't use the DHCP (Dynamic Host Configuration protocol) with a static IP address. The following macros in *lwipopts.h* must be defined as follows:

```
/* SO_REUSE ==1: Enable SO_REUSEADDR option */
#define SO_REUSE           1

#ifndef LWIP_DHCP
#define LWIP_DHCP          0
#endif

/* ---------- DNS options ---------- */
#ifndef LWIP_DNS
#define LWIP_DNS           1
#endif

/* ---------- IGMP options ---------- */
/* LWIP_IGMP==1: Turn on IGMP module. */
#ifndef LWIP_IGMP
#define LWIP_IGMP          1
#endif
```

The default lwIP package in the SDK release doesn't support PTP. The ENET initializing function for the lwIP does not involve any 1588 timer functions. The code update for the *ethernetif.h* and *ethernetif.c* files covers mainly the ENET 1588 timer routines, such as initializing the 1588 timer, enabling the timer channel output compare function for test, setting/getting the time, adjusting the 1588 timer frequency, and getting the timestamp of the transmit/receive frames. All codes related to PTP are enclosed by the *#if LWIP_TPT* and *#endif* pair.

This code snippet must be added into the *ethernetif.h* file to declare the routines of the 1588 timer:

```
#if LWIP_PTP
#include "lwip_ptp.h"

#define ENET_NANOSECOND_ONE_SECOND      10000000000U
#define PTP_AT_INC                      (ENET_NANOSECOND_ONE_SECOND/PTP_CLOCK_FRE_RT)

void ethernet_ptptime_settime(enet_ptp_time_t *timestamp);
void ethernet_ptptime_gettime(enet_ptp_time_t *timestamp);
void ethernet_ptptime_adjfreq(int32_t ppb);
err_t enet_get_rxframe_time(enet_ptp_time_data_t *ptpTimeData);
err_t enet_get_txframe_time(enet_ptp_time_data_t *ptpTimeData);
#endif
```

The above functions are implemented in the *ethernetif.c* file. The ENET 1588 timer-initializing function *ethernet_ptptime_init ()* is implemented and called before returning from the *enet_init()* function. The *ethernet_ptptime_enablepps()* function is implemented to enable the timer channel output compare function for test purposes and called in the *ethernet_ptptime_init ()* function according to the passed parameter.

This is the code of the *ethernet_ptptime_enablepps()* function:

```
static void ethernet_ptptime_enablepps(struct ethernetif *ethernetif,
                                       enet_ptp_timer_channel_t tmr_ch)
{
    uint32_t next_counter = 0;
    uint32_t tmp_val = 0;

    /* clear capture or output compare interrupt status if have. */
    ENET_Ptp1588ClearChannelStatus(ethernetif->base, tmr_ch);

    /* It is recommended to double check the TMODE field in the
     * TCSR register to be cleared before the first compare counter
     * is written into TCCR register. Just add a double check. */
    tmp_val = ethernetif->base->CHANNEL[tmr_ch].TCSR;
    do {
        tmp_val &= ~(ENET_TCSR_TMODE_MASK);
        ethernetif->base->CHANNEL[tmr_ch].TCSR = tmp_val;
        tmp_val = ethernetif->base->CHANNEL[tmr_ch].TCSR;
    } while (tmp_val & ENET_TCSR_TMODE_MASK);

    tmp_val = (ENET_NANOSECOND_ONE_SECOND >> 1);

    ENET_Ptp1588SetChannelCmpValue(ethernetif->base, tmr_ch, tmp_val);

    /* Calculate the second the compare event timestamp */
    next_counter = tmp_val;

    /* Compare channel setting. */
    ENET_Ptp1588ClearChannelStatus(ethernetif->base, tmr_ch);

    ENET_Ptp1588SetChannelOutputPulseWidth(ethernetif->base, tmr_ch, false, 4, true);

    /* Write the second compare event timestamp and calculate
     * the third timestamp. Refer the TCCR register detail in the spec.*/
    ENET_Ptp1588SetChannelCmpValue(ethernetif->base, tmr_ch, next_counter);
    /* Update next counter */
    ethernetif->handle.ptpNextCounter = next_counter;
}
```

This code is the ENET 1588 timer-initializing function *ethernet_ptptime_init ()* and its related memories:

```
static struct ethernetif * ptp_ethernetif  = NULL;

/* Buffers for store receive and transmit timestamp. */
//static sys_mutex_t ptp_mutex;
enet_ptp_time_data_t g_rxPtpTsBuff[ENET_RXBD_NUM];
enet_ptp_time_data_t g_txPtpTsBuff[ENET_TXBD_NUM]

static void ethernet_ptptime_init(struct ethernetif *ethernetif, uint32_t ptp_clk_freq,
                                  bool pps_en, enet_ptp_timer_channel_t tmr_ch)
{
  enet_ptp_config_t ptp_cfg;

  assert(ethernetif);
  ptp_ethernetif = ethernetif;

  /* Config 1588 */
  memset(&ptp_cfg, 0, sizeof(enet_ptp_config_t));
  ptp_cfg.channel = tmr_ch;
  ptp_cfg.ptpTsRxBuffNum = ENET_RXBD_NUM;
  ptp_cfg.ptpTsTxBuffNum = ENET_TXBD_NUM;
  ptp_cfg.rxPtpTsData = &g_rxPtpTsBuff[0];
  ptp_cfg.txPtpTsData = &g_txPtpTsBuff[0];
  ptp_cfg.ptp1588ClockSrc_Hz = ptp_clk_freq;
  ENET_Ptp1588Configure(ptp_ethernetif->base, &ptp_ethernetif->handle, &ptp_cfg);

  if (true == pps_en)
  {
    ethernet_ptptime_enablepps(ptp_ethernetif, tmr_ch);
  }
  else
  {
    ENET_Ptp1588SetChannelMode(ptp_ethernetif->base, tmr_ch, kENET_PtpChannelDisable, false);
  }
}
```

The syntax in red is used to call *ethernet_ptptime_init ()* in *enet_init()*:

```
static void enet_init(struct netif *netif, struct ethernetif *ethernetif,
                      const ethernetif_config_t *ethernetifConfig)
{
    ……
#if LWIP_PTP
    /* It's time to initialize the IE1588 function of ethernet */
    ethernet_ptptime_init(ethernetif, PTP_CLOCK_FRE_RT, PTP_TEST_APP_ENABLE,
                          PTP_TEST_APP_CHANNEL);
#endif
    ENET_ActiveRead(ethernetif->base);
}
```

The *ethernet_ptptime_adjfreq()* function adjusts the 1588 timer frequency by setting the non-zero correction counter wrap-around value in the *ENET_ATCOR* register to define by how many timer clock

cycles to correct the 1588 timer's time. The correction increment value is set in the *INC_CORR* field in the *ENET_ATINC* register. The value of *INC_CORR* is larger than the value in the *INC* field to speed up the 1588 timer; The value of *INC_CORR* is lower than the value in the *INC* field to slow down the 1588 timer.

This is the code of *ethernet_ptptime_adjfreq()*:

```
void ethernet_ptptime_adjfreq(int32_t incps)
{
    int32_t neg_adj = 0;
    uint32_t corr_inc, corr_period;

    assert(ptp_ethernetif);

    /*
     * incps means the increment rate (nanseconds per second)by which to
     * slow down or speed up the slave timer.
     * Positive ppb need to speed up and negative value need to slow down.
    */

    if (0 == incps)
    {
        ptp_ethernetif->base->ATCOR &= ~ENET_ATCOR_COR_MASK; /* Reset PTP frequency */
        return;
    }

    if (incps < 0)
    {
        incps = - incps;
        neg_adj = 1;
    }

    corr_period = (uint32_t)PTP_CLOCK_FRE_RT / incps;

    /* neg_adj = 1, slow down timer, neg_adj = 0, speed up timer */
    corr_inc = (neg_adj) ? (PTP_AT_INC - 1) : (PTP_AT_INC + 1);

    ENET_Ptp1588AdjustTimer(ptp_ethernetif->base, corr_inc, corr_period);
}
```

The *ethernet_ptptime_settime()*, *ethernet_ptptime_gettime()*, *enet_get_rxframe_time()*, and *enet_get_txframe_time()* functions are implemented to wrap the *ENET_Ptp1588GetTimer()*, *ENET_Ptp1588SetTimer()*, *ENET_GetRxFrameTime()*, and *ENET_GetTxFrameTime()* functions in the *ethernetif.c* file.

5.3. PTPd porting on FreeRTOS

The default PTP daemon (PTPd) source code exists for the FreeBSD, NetBSD, Mac OS X, and Linux OS systems. To port it on FreeRTOS with the lwIP and SDK drivers for the EVK-MIMXRT1050, the operation system-related code, network-related code, and hardware timestamping code must be ported or added. The ported work covers the PTPd tasks under the FreeRTOS, system time routines, system services, software timer, interaction with the network socket, and minor modification of the PTP protocol. The trivial code modifications required by the IAR Embedded Workbench for ARM during the compiling and linking are not discussed here.

Although the code in the files in the *ptpd/src* folder are common to the PTPd application, some files must be updated for the PTPd to work on the FreeRTOS system. The original *main()* function in the *ptpd.c* file is changed to *ptpd_thread()*, which is created as a FreeRTOS task. The original command line parameters are removed to simplify the demo functions, except for the variable to denote the master or the slave. This variable's value is passed by the parameter while creating a FreeRTOS task.

Another significant modification in the PTPd common code is in the *protocol.c* file. The default PTPd application runs as a real network node within a Unix-style system. The device can receive the frames of event messages sent by itself because they are sent using the UDP/IP multicast messages. The *Follow_Up* or *Pdelay_Resp_Follow_Up* message is sent after the device receives the corresponding *Sync* or *Pdelay_Resp* event message sent by itself in the original protocol source code. When this demo runs with a point-to-point connection, the device does not receive the event message sent by itself. The code must be modified to send these two follow-up messages when the corresponding event message is sent. As a result of this change, the *netSelect()* function must be called with a specific timeout value to replace the original NULL (no timeout) that blocks the *select()* function waiting for the file descriptor ready indefinitely.

The files in the *ptpd/src/dep* folder are port-specific source codes and dependent on the operating system, TCP/IP stack, and hardware platform. The main changes involve the *net.c*, *startup.c*, *sys.c*, and *timer.c* files. Their names are suffixed by "*_mcu*" to distinguish them from the original files.

The FreeRTOS provides the software timer functionality when setting *configUSE_TIMERS* to 1 in the *FreeRTOSConfig.h* file. The modification to the *timer_mcu.c* file includes these two functions:

```
void catch_alarm(TimerHandle_t xTimer)
{
  (void)xTimer;

  elapsed++; /* be sure to NOT call DBG in asynchronous handlers! */
}

void initTimer(void)
{
  TimerHandle_t xptpTimer;

  xptpTimer = xTimerCreate("ptp_timer", pdMS_TO_TICKS(MS_TIMER_INTERVAL), pdTRUE, NULL,
catch_alarm);

  if(xptpTimer != NULL)
  {
    xTimerStart(xptpTimer, portMAX_DELAY);
  }
}
```

The *sys_mcu.c* file has time-related routines to provide interfaces to the low-level hardware timer that is synchronized in the demo. Most of them call the functions described in Section 5.2, "lwIP TCP/IP porting update".

The *adjFreq()* function code is as follows:

```
Boolean adjFreq(Integer32 adj)
{
    if (adj > ADJ_FREQ_MAX)
        adj = ADJ_FREQ_MAX;
    else if (adj < -ADJ_FREQ_MAX)
        adj = -ADJ_FREQ_MAX;
    ethernet_ptptime_adjfreq(adj);
    return TRUE;
}
```

There are two sleep functions for the current thread to enter a dormant state that are implemented using the FreeRTOS *vTaskDelay()* function. The other changes just remove the code for the information output and/or log file.

```
Boolean nanoSleep(TimeInternal * t)
{
    TickType_t time;

    time = pdMS_TO_TICKS(t->seconds * 1000 + t->nanoseconds / 1000000);
    vTaskDelay(time);
    return TRUE;

}

void milliSleep(int milli_seconds)
{
    TickType_t time;

    time = pdMS_TO_TICKS(milli_seconds);
    vTaskDelay(time);
}
```

The *startup_mcu.c* file removes all OS-related signal functions and the command line parameter parsing code. The *ptpd_init()* function is added to create a FreeRTOS task for the PTP daemon application.

The timestamp of the transmit frame in the original code is provided by the operating system and the timestamp of the received frame can be extracted from the received data after calling the *recvmsg()* function, which is supported by the Linux-style operating systems. This demo uses the hardware timestamping feature in the ENET 1588 timer. These codes must be ported to the FreeRTOS and the ENET 1588 timer on the i.MX RT1050 MCU. The code in the *net_mcu.c* file provides the ported functions.

The *findIface()* function directly returns the default network interface used in the lwIP and implemented as follows:

```
#define netGetDefaultNetif() (netif_default)

UInteger32 findIface(Octet * ifaceName, UInteger8 * communicationTechnology,
                     Octet * uuid, NetPath * netPath)
{
    struct netif * iface;
    (void) communicationTechnology;
    (void) netPath;
    iface = netGetDefaultNetif();

    memcpy(uuid, iface->hwaddr, iface->hwaddr_len);
    memcpy(ifaceName, iface->name, sizeof (iface->name));
    return iface->ip_addr.addr;
}
```

Because the ported code doesn't use *recvmsg()* to have timestamps available, the *netInitTimestamping()* function is not called in *netInit()* or implemented as a dump function just returning TRUE.

The ENET driver of the i.MX RT 1050 MCU provides two APIs to query the timestamp of the frame of the event message that is transmitted or received at a time. To get the timestamp, the ENET PTP message data and the timestamp data defined by the *enet_ptp_time_data_t* type must be packed from the buffer that contains the received or sent event message. The *netPackPtpData()* function is added for this task and listed explicitly here as follows:

```
static void netPackPtpData(Octet * buf, enet_ptp_time_data_t *pptpTimeData)
{
       pptpTimeData->messageType = (*(Enumeration4 *) (buf + 0)) & 0x0F;
       pptpTimeData->sequenceId = flip16(*(UInteger16 *) (buf + 30));
       pptpTimeData->version = (*(UInteger4 *) (buf + 1)) & 0x0F;
       memcpy(pptpTimeData->sourcePortId, (buf + 20), 10);
}
```

The *recv()* socket function replaces *recvmsg()* in *netRecvGeneral()* to read a frame of the general message. This is the code of the *netRecvGeneral()* function implementation:

```
ssize_t netRecvGeneral(Octet * buf, TimeInternal * time, NetPath * netPath)
{
       ssize_t ret;

       ret = recv(netPath->generalSock, buf, PACKET_SIZE, MSG_DONTWAIT);
       if (ret <= 0) {
              if (errno == EAGAIN || errno == EINTR)
                     return 0;
              return ret;
       }

       return ret;
}
```

The netRecvEvent() function reads a frame of the event message and returns its timestamp provided by the low-level ENET driver.

This is the code of its implementation:

```
ssize_t netRecvEvent(Octet * buf, TimeInternal * time, NetPath * netPath)
{
	ssize_t ret;
	enet_ptp_time_data_t ptpTimeData;

	getTime(time); /* Current time for timestame in case of reading from driver failed. */
	ret = recv(netPath->eventSock, buf, PACKET_SIZE, MSG_DONTWAIT);
	if (ret <= 0) {
		if (errno == EAGAIN || errno == EINTR)
			return 0;
		return ret;
	}

	if (!time) {
		ERROR("null receive time stamp argument\n");
		return 0;
	}

	netPackPtpData(buf, &ptpTimeData);
	if(!enet_get_rxframe_time(&ptpTimeData)){
		time->nanoseconds = ptpTimeData.timeStamp.nanosecond;
		time->seconds = (Integer32)ptpTimeData.timeStamp.second;
	}
	return ret;
}
```

Both *netSentEvent()* and *netSentPeerEvent()* send the frame of the corresponding event message and return the frame's timestamp. The default implementation doesn't support hardware timestamping. To enable hardware timestamping, the *netSentEvent()* and *netSentPeerEvent()* functions add another input parameter of pointer to the buffer that contains the returned timestamp.

This code snippet shows the code added into the *netSentEvent()* and *netSentPeerEvent()* functions in red:

```
ssize_t netSendEvent(Octet * buf, UInteger16 length, TimeInternal * time, NetPath * netPath,
Integer32 alt_dst)
{
    ……
    enet_ptp_time_data_t ptpTimeData;
    ……

    getTime(time); /* Current time for timestame in case of reading from driver failed. */
    netPackPtpData(buf, &ptpTimeData);
    if(!enet_get_txframe_time(&ptpTimeData)){
        time->nanoseconds = ptpTimeData.timeStamp.nanosecond;
        time->seconds = (Integer32)ptpTimeData.timeStamp.second;
    } else {
        /* suspend process 1 millisecond to make sure current frame was sent by enet */
        milliSleep(1);
        /* Try to read timestamp again */
        if(!enet_get_txframe_time(&ptpTimeData)){
            /* return the timestamp gotten from driver */
            time->nanoseconds = ptpTimeData.timeStamp.nanosecond;
            time->seconds = (Integer32)ptpTimeData.timeStamp.second;
        }
    }
    return ret;
}
```

5.4. FreeRTOS tasks and board configuration

There are three task threads created using FreeRTOS in the demo application:

- *stack_init* task—created in the main function. This task initializes the lwIP TCP/IP stack and the static IP address setting, netmask configuration, gateway address configuration, MAC address configuration, and Ethernet hardware initializing and starts the *PTPd* task. The task then deletes itself by calling the *vTaskDelete()* function after initializing the *lwIP* and *PTPd* tasks.
- *tcpip_thread* task—created by the the *stack_init* task during the TCP/IP initialization and runs the main *lwIP* task to access the lwIP core functions.
- *ptpd_thread* task—created by the the *stack_init* task and runs the PTP daemon application. The parameter passed while creating this task denotes either the master or the slave.

The demo enables the 1588 timer's 1-channel output compare function. Its output signal is asserted according to the configuration while the output compare event happens. The 1588 timer channel 3 is used to generate the output compare event in this demo. The output signal is a routine to the *GPIO_AD_B1_02* pin.

This syntax configures the *GPIO_AD_B1_02* as *ENET_1588_EVNT2_OUT* (output signal of channel 3):

```
/* GPIO_AD_B1_02 is configured as 1588_EVENT2_OUT */
IOMUXC_SetPinMux(IOMUXC_GPIO_AD_B1_02_ENET_1588_EVENT2_OUT, 0U);

/* GPIO_AD_B0_12 PAD functional properties */
IOMUXC_SetPinConfig(IOMUXC_GPIO_AD_B1_02_ENET_1588_EVENT2_OUT, 0x10B0u);
```

The clock of the 1588 timer is generated from *ref_enetpll2* by the Ethernet PLL that must be enabled. The Ethernet PLL is initialized as follows:

```
void BOARD_InitModuleClock(void)
{
    const clock_enet_pll_config_t config = {true, false, true, 1, 1};
    CLOCK_InitEnetPll(&config);
}
```

The remaining board-specific initialization code is the same as the *enet_rxtx_ptp1588* example in the *<sdk_install_dir>/boards/evkmimxrt1050/driver_examples/enet/txrx_ptp1588_transfer* folder.

6. Running the IEEE1588 demo

This section describes how to set up the 1588 demo using the EVK-MIMXRT1050 board and the demo software described in the previous sections.

6.1. Hardware setup

Two EVK-MIMXRT1050 boards must be used and connected to each other to set up the test hardware. The demo has a point-to-point configuration where two boards are connected directly using the crossover Ethernet cable. This is a simple type of connection often used to evaluate the system accuracy and the overall performance. The point-to-point configuration using two EVK-MIMXRT1050 boards is shown in Figure 10. There is no specific jumper setting for the test.

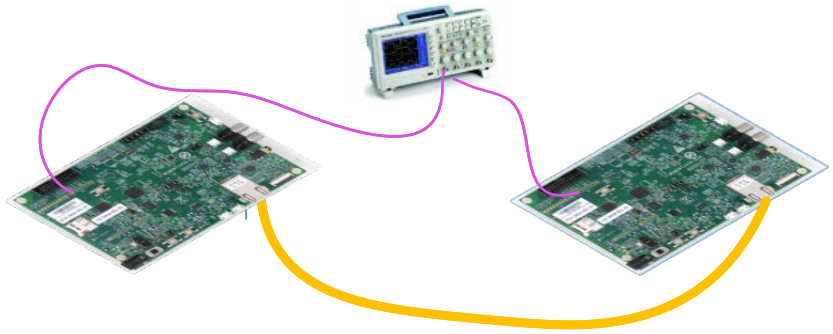

Figure 10. Back-to-back demo configuration

For detailed information on how to use the EVK-MIMXRT1050 and set its jumpers, see the *MIMXRT1050 EVK Board Hardware User's Guide* (document MIMXRT1050EVKHUG).

6.2. Measuring clock synchronicity

This demo can generate a Pulse-Per-Second (PPS) signal to measure the synchronicity of the clocks between the master and the slave. The PPS signal is generated directly from the 1588 timer channel 3 and configured to output through the *GPIO_AD_B1_02* pin. This GPIO signal is routed to the *J22-7* pin of the Arduino interface.

To measure and compare the PPS signals from two boards, attach two oscilloscope probes to the *J22-7* pins on two EVK-MIMXRT1050 boards, respectively. The two boards are powered with a time interval in the test. The oscilloscope shows that the slave PPS signal is getting closer and closer to the master PPS signal and the offset converges to vary between a range of four clock cycles of the 1588 timer.

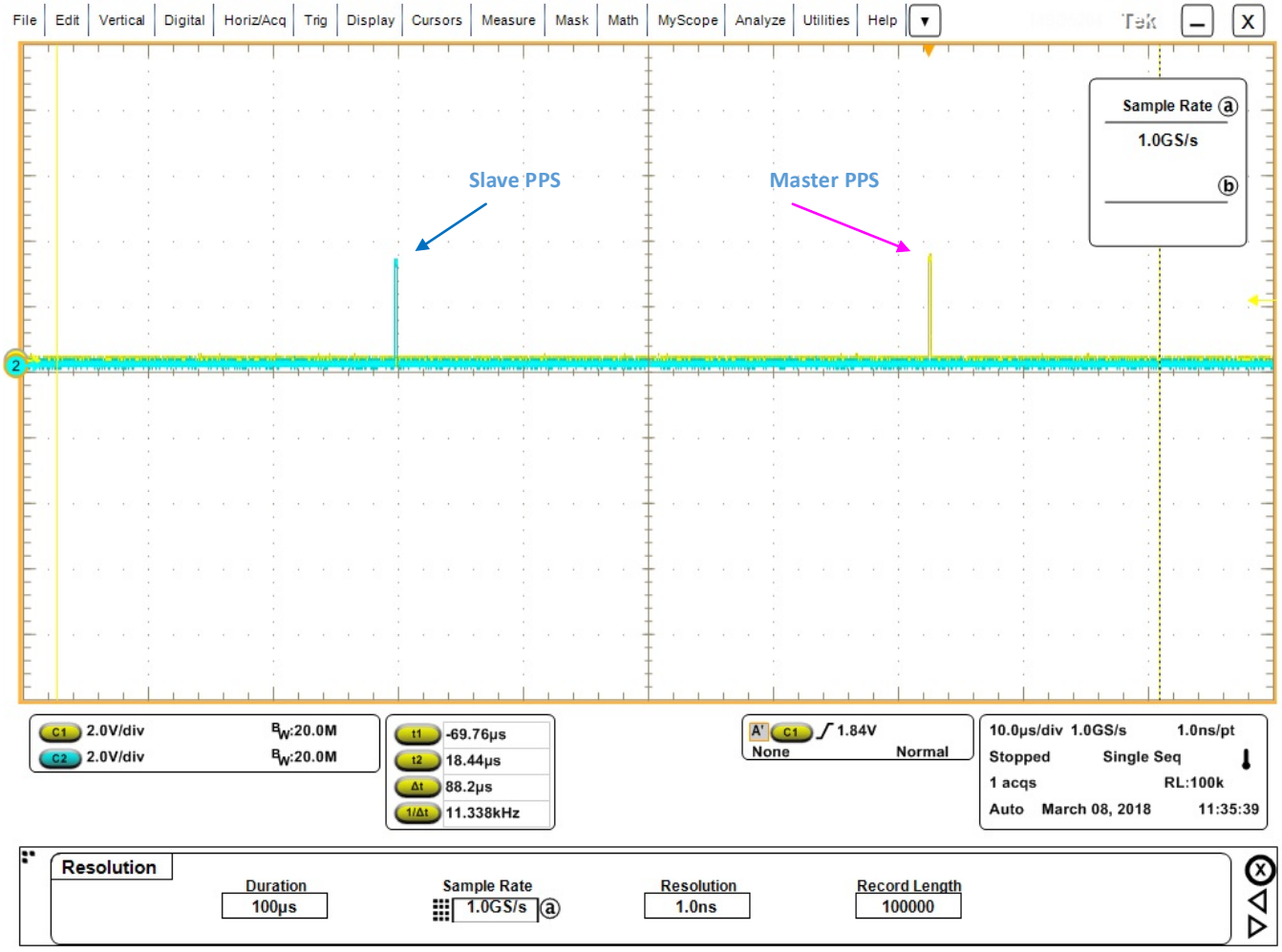

Figure 11. PTP startup—PPS distance between master and slave

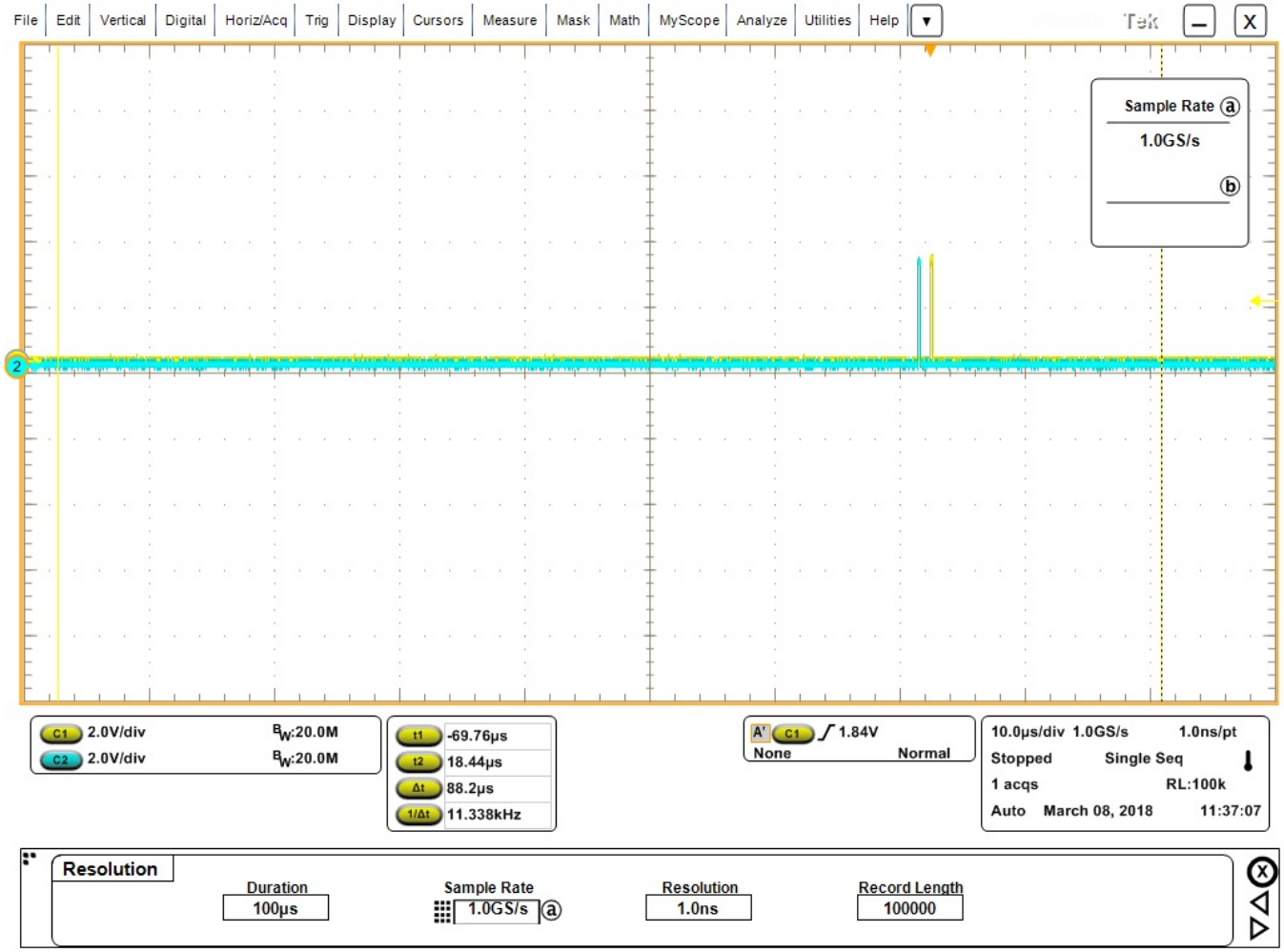

Figure 12. PTP synchronizing—slave PPS closing to master PPS

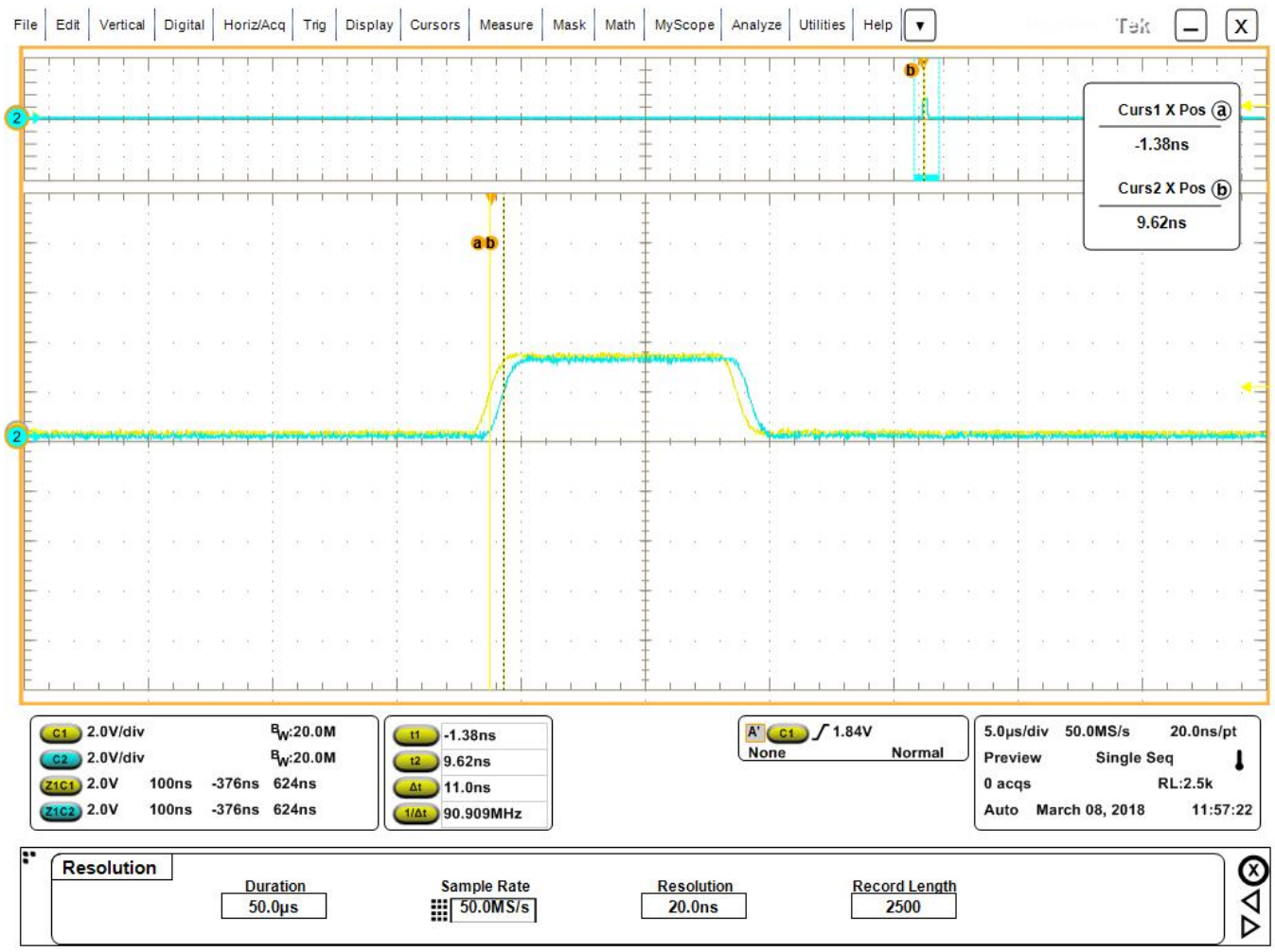

Figure 13. PTP synchronized—converged PPS distance between master and slave

7. Conclusion

This application note describes the IEEE 1588 Precision Time Protocol demo application based on the open-source PTP daemon, FreeRTOS, lwIP TCP/IP stack, SKD for i.MX RT1050, and the MIMXRT1050 Evaluation Kit (EVK-MIMXRT1050). This demo can be easily ported to other processors from the i.MX RT series with the FreeRTOS and lwIP TCP/IP stack support.

The demo system is targeted at applications that require precise clock synchronization between devices with the accuracy in the sub-microsecond range.

8. Acronyms and abbreviations

Table 1. Acronyms and abbreviations

Term	Meaning
API	Application Program Interface
BMC	Best Master Clock
DHCP	Dynamic Host Control Protocol
DNS	Domain Name System
ENET	10/100-Mbps Ethernet MAC
GPIO	General Port Input Output
ICMP	Internet Control Message Protocol
IGMP	Internet Group Management Protocol
MAC	Media Access Control
PPS	Pulse-Per-Second
PTP	Precision Time Protocol
PTPd	PTP daemon
RTOS	Real-Time Operation System
SDK	Software Development Kits
TCP	Transmission Control Protocol
UDP	User Datagram Protocol

How to Reach Us:

Home Page:
nxp.com

Web Support:
nxp.com/support

Document Number: AN12149
Rev. 0
03/2018